

# EGB320 Final Report: Mobility Subsystem

Darcy Stewart  
N11977965  
Group 3

November 2, 2025

## Table Of Contents

|          |                                               |          |
|----------|-----------------------------------------------|----------|
| <b>1</b> | <b>Problem Description</b>                    | <b>3</b> |
| <b>2</b> | <b>Background Research</b>                    | <b>4</b> |
| <b>3</b> | <b>Design</b>                                 | <b>5</b> |
| 3.1      | Track Selection . . . . .                     | 5        |
| 3.2      | Motor Selection . . . . .                     | 5        |
| 3.3      | Motor Controller Selection . . . . .          | 6        |
| 3.4      | Power Management System Selection . . . . .   | 6        |
| 3.5      | Motor Controller Programming . . . . .        | 6        |
| 3.6      | Communication Code For Raspberry Pi . . . . . | 8        |
| 3.6.1    | Set Speed . . . . .                           | 8        |
| 3.6.2    | Drive distance . . . . .                      | 8        |
| 3.6.3    | Turn degrees . . . . .                        | 8        |
| 3.7      | Chassis Design . . . . .                      | 8        |
| 3.8      | Design Validation . . . . .                   | 9        |
| <b>4</b> | <b>Conclusion</b>                             | <b>9</b> |

# 1 Problem Description

The problem set forward by the client was that shelves needed restocking, and it was believed that a robotic system would provide a more cost-effective method for shelf restocking. To evaluate the suitability of a robotic warehousing system a Technology Readiness Level 3(TRL3) prototype. This prototype was designed to work in a scaled down mock warehouse with shelves, items and obstacles that would simulate the major factors around navigating and moving stock around a grocery store.

For this prototype some adjustments were made to allow for more streamlined development. The first major adjustment was to have each different element in the store be represented by an object of a different color and shape as shown below:

| Element                          | Representation   |
|----------------------------------|------------------|
| Obstacle                         | Green Colouring  |
| Item                             | Orange Colouring |
| Aisle and Picking Station Marker | Black Colouring  |
| Shelves                          | Blue Colouring   |
| Picking Station                  | Yellow Colouring |

These adjustments were made to allow for a more simplified vision system which in turn would allow for rapid development. Further prototypes will build on this model to allow for more complex vision processing. Along with this the shelf heights were the same for all aisles. Allowing for simplified item placement and navigation code as no matter the bay the heights will always remain the same. The final simplification is the distance between the picking stations, this distance allows for a lower level accuracy being required to pick up an item.

With these simplifications in mind 4 separate subsystems were identified that would be designed to integrate to a final product. The intention behind this was to further simplify the system through having each system capable of operating separately with methods of communication to allow for integration. This allowed for the 4 subsystems to be designed in parallel and setbacks in one element of the robot would not necessarily set back the overall progress of the robot.

Following this the requirements for each subsystem were identified, for mobility these requirements were as follows:

- Turn to a bearing on a single point in a given direction
- Turn at a set speed in a given direction
- Move forward and backward at a set speed
- Move forward and backward a set distance
- Have a chassis that is capable of carrying all other subsystems
- Have a power management system that can power all other subsystems

With these requirements identified background research was conducted to determine the most suitable solutions.

## 2 Background Research

Research was conducted into various methods for robotic mobility used in similar applications in industries. Four major methods of mobility were identified:

- Wheels
- Tracks
- Legs
- Mecanum Wheels

An investigation into the pros and cons of each system was conducted to allow for a clear evaluation of each option's suitability. The results of this investigation were compiled into the table seen below.

| Feature                                          | Wheels | Tracks | Legs | Mecanum Wheels |
|--------------------------------------------------|--------|--------|------|----------------|
| Can spin in place                                | ✗      | ✓      | ✓    | ✓              |
| Reliably stable without extra sensors            | ✓      | ✓      | ✗    | ✓              |
| Requires 2 motors or less                        | ✓      | ✓      | ✗    | ✗              |
| Can easily have speed measured in all directions | ✓      | ✓      | ✗    | ✗              |
| Easily programmable                              | ✓      | ✓      | ✗    | ✗              |

As shown in the table above it was decided that tracks would be investigated as the primary option for mobility. This was due to its ability to navigate varying terrain with small obstacles, while maintaining a simple drive system to ensure a reliable outcome. The Treads also have a high amount of surface friction due to the large surface area in contact with the ground. However, there were concerns about the reliability of tracks as consulting an expert on their usage in the field found that they can be prone to slipping off the retainer system over time. This concern was minimal as the environment that the tracks were being deployed in varied greatly from the environment of a warehouse.

Once the research into the mobility system was done it was crucial to determine a method in which the motors will be controlled. Initially it was thought that the Raspberry Pi utilized for the vision and navigation subsystems would also be used to control the motors. This plan was found to have issues one research found that the Raspberry Pi GPIO pins were only capable of 50mA total for the entire pin header. This was a major concern as the motors can easily draw over 5 times this amount under a standard load.

Further research was conducted and found that the majority of robots that involved a mobility subsystem made use of one or multiple motor controllers. These motor controllers take input signals from a central control system and take power directly from the power management system to drive the motors. This bypasses the 50mA current limit of the GPIO pins and allows for stronger motors to be used.

Once this had been decided the next significant piece of research was to determine a power management system that would be capable of supplying the robot and all of its peripherals with power. As the robot's main power supply was an 8.4V Lithium-ion battery it's voltage would need to be regulated down to the required 5V input for the Raspberry Pi. Multiple options were considered for this regulator component, the first being a simple 5V linear regulator due to its simplicity for implementation. This idea was moved past due to concerns that the with the efficiency of the regulator. Following this research was conducted into the feasibility of a switching regulator. Research found that switching regulators are significantly more efficient in their power regulation, however they add high levels of noise to the circuit which can affect signals in the system. This however wasn't deemed to be a major issue as there were no components that are highly susceptible to noise. With research conducted into all elements required for the mobility system it was then time to begin the actual design and component selection for the robot.

## 3 Design

The design of the subsystem had multiple individual sections that had to be developed sequentially. These sections included, track selection, motor selection, motor controller selection, power management system design, motor controller selection and motor controller programming. Each section had their own challenges and needs that had to be evaluated and designed around. Each section and its design considerations are detailed below.

### 3.1 Track Selection

The process of selecting tracks was relatively straightforward as not many of the accessible suppliers that stocked tracks were accessible within a time period that would be suitable for the project timeline. This was a major deciding factor, as without a functional mobility system the testing of navigation on the actual robot could not commence. With this in mind local Australian based suppliers became the preferable option. With this limiting factor it was found that Robot Gear Australia had the widest range of options. These options mainly consisted of Pololu Track sets which came in different size sprockets and track belts.

It was decided that the 30 tooth track set was the most suitable, this was mainly due to the sprocket shaft spacing of 85mm. This combined with the sprocket radius of 35mm gives a total track length of 120mm, 60% of the maximum length of 200mm. This length provides stability from rotational forces caused by the robot driving forward and reverse with a mass held at a height. With a decision made on the track set to be used the next in the design process was motor selection.

### 3.2 Motor Selection

With the tracks selected some Pololu N20 motors were trialed to drive the system. The main reason behind this was that they were already owned and compatible with the track set. These motors were 50:1 N20 DC motors with magnetic encoders and a max output torque of 400gcm. These trial motors were fitted to a baseplate and powered by an exiting motor driver hat designed to integrate with the Raspberry Pi. Testing found that while the motors had sufficient power output to drive the Raspberry Pi and the mounting baseplate they were unable to move the robot up the necessary inclines with the total predicted system weight. As such output torque in a small format became the priority for motor selection.

With output torque becoming the priority for motor selection it became necessary to calculate the minimum torque required to achieve the required movements. The torque required was calculated using the formula:

$$T_m = \mu_s \times m \times g \times radius$$

The torque was calculated for the system using an estimated robot mass of 900 grams with a coefficient of friction of 0.45 for carpet. With an applied factor of safety of 1.2 the estimated torque required was found to be 850gcm. With this value known research was conducted to find an N20 motor that would be capable of producing the required levels.

Other parameters influenced motor selection on top of this. The biggest deciding factor for the motor was the requirement for magnetic encoders to be present or a third party option that is compatible. This was determined to be necessary as it is the most common and well documented method for measuring and adjusting motor output speed, a feature required for accurate driving. With these requirements local suppliers were searched yielding the WaveShare N20 1:150 encoded motors. The WaveShare motors had the same form factor as the trial motors, meaning that a redesign wasn't necessary, and 1.1kgcm of torque, all with an encoder. These motors were also to most cost-effective option at \$15 AUD a piece.

The selection of the new motors brought an additional complication to the design as they required 1.1A stall current. This was a concern as the pre-existing motor driver hat was only capable of outputting 0.5A per motor. As such with the pre-existing motor driver hat the new motors would not be able to be used to their

full advantage. To address this issue a search for a new motor driver was deemed necessary.

### 3.3 Motor Controller Selection

The selection for the motor controller had an added level of complexity as it was decided that item collection subsystem would use encoded motors to operate and as such would require a motor driver with the same capabilities. To minimize the footprint of the motor controllers that would be needed it was decided that both subsystems should be controlled by the same motor controller. This would minimize the amount of connection points between the motor controller and as such limit the amount of potential failure points in the system.

The number of 4 channel encoder capable motor controllers is very limited. There were many 2 channel options that were investigated as it was believed that there were no other options. Eventually two options were found that offered met the 4 channel requirements the M5Stack Encoded Motor Controller and the Pimoroni Pico Motor 2040 motor controller. In terms of electrical functionality the two were equal with each other and the M5Stack sat at a much better price point, however when reviewing the code libraries for each it was found that the Pimoroni motor controller had a much better documented library with extensive examples. This was the major deciding factor as the better example libraries would allow for quicker and better writing of code to achieve the desired goal. As such it was decided that the Pimoroni Pico Motor 2040 would be the motor driver. With this determined the final part of component selection required was the power management system

### 3.4 Power Management System Selection

The first stage of designing the Power Management system was to estimate the power draw caused by the system. The Raspberry Pi is listed to draw 5A of power at 5V while running all the motors in the robot at once would add up to 1.1A a piece. As such the initial power requirement estimate was calculated to be 9.4A of power draw, this was rounded up to 12A to give a safe thermal ceiling to ensure that components didn't overheat. The ideal component to handle this was found in the AOZ2295QI-05 DCDC Buck Regulator and a prototype board was developed.

This initial prototype board was found to have several issues relating to component placement that resulted in the regulator burning out. At this time it was also advised that the battery that was required to safely power the robot had an internal current limiting circuit that limited output to 6A. This resulted in a major reevaluation of the design to investigate whether the current design was still suitable. Research found that the Raspberry Pi was capable of running in a low power mode that drew only 3A and with careful navigation code only 2 motors would be driven at once, thus determine out current as still being feasible.

This new current limitation meant that there was a lot more in terms of options for premade voltage regulators. This lead to the decision to use the DF Robot 25W adjustable voltage regulator. The main reason behind the decision to use this voltage regulator was that it not only had an adjustable regulated output, but it also had a raw output. This allowed for the motors controlled by the motor driver to be powered directly by the battery and as such could pull however much voltage was required. Allowing for only the microcontroller unit of the motor driver to rely on the regulated voltage.

### 3.5 Motor Controller Programming

The motor controller was programmed in C++, mainly to utilize the object-oriented nature of the language and highly optimized machine code. The Pimoroni Pico libraries had motor objects where all motor values could be saved to the motor info, this was the same for the encoders. Following this a PID controller was calibrated for the motors, this had to be done with position and speed PID models separately as the different functions required different models. The calibration process utilized was to start with proportional element and increase until there were oscillations and halve the value before moving on to the integral and derivate elements with the same process.

Once the PID model had been calibrated for each motor separately the next stage was to set up a method of communication between the Raspberry Pi and Pimoroni Pico. This was achieved through using the `stdlib.h` library that allows for easy setup of serial communication of universal serial bus (USB). Serial communication was chosen due to the ease of access to the USB port and the many Python and C++ libraries that allow for easy use. Time had to be spent building a communication protocol that would be followed using this. The communication protocol went as follows:

| Character     | Velocity Control                                                                     | Positional Control                                                                        |
|---------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| M             | Designates that a motor state is going to be changed                                 | Designates that a motor state is going to be changed                                      |
| A, B, C, D    | Designates which motor is affected                                                   | Designates which motor is affected                                                        |
| 1,0           | 0 sets up for velocity control                                                       | 1 sets up positional control                                                              |
| integer (0→9) | Tells the controller the velocity in $\text{value} \times 1$ rotations per second    | Tells the controller the number of rotations to do in $\text{value} \times 10$ rotations  |
| integer (0→9) | Tells the controller the velocity in $\text{value} \times 0.1$ rotations per second  | Tells the controller the number of rotations to do in $\text{value} \times 1$ rotations   |
| integer (0→9) | Tells the controller the velocity in $\text{value} \times 0.01$ rotations per second | Tells the controller the number of rotations to do in $\text{value} \times 0.1$ rotations |
| 1,0           | Sets direction                                                                       | Sets direction                                                                            |

This communication protocol was read through detecting an incoming message and saving each character of the message to into a character array that was then interpreted. Once the message was read a motor control function was used to interpret the message and execute the desired functions. Once this was completed the program would check state of each motor and make adjustments according to the PID system to ensure that all operations are being completed as desired. For positional control if the Robot has moved the set distance the motor controller will also send a signal back to Raspberry Pi to allow it to move on in the program. The workflow of the Microcontroller program is show in figure 1.

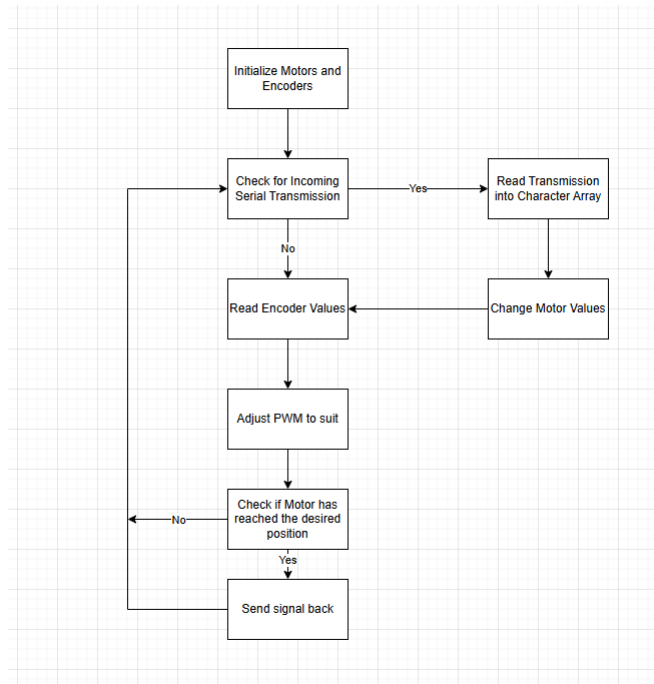


Figure 1: Figure 1: Microcontroller code flow chart

## 3.6 Communication Code For Raspberry Pi

The next stage was to develop the code that Raspberry Pi would use to interact with the motor controller. The main purpose of the code was the convert the desired distances, speeds and angle of rotation to the necessary values for the communication protocols. The breakdown for each mobility function is detailed below:

### 3.6.1 Set Speed

The set speed function takes a value of direction and speed. With direction being the characters r and f while speed is measured in m/s. The function takes the desired speed and converts the meters to revolutions before converting the floating point number to a 3 digit value according to the motor controller communication protocol. The motors responsible for moving the robot then have the information sent through according to the communication protocol and the function terminates.

### 3.6.2 Drive distance

The drive distance function takes a value of direction and distance measured in mm. The function converts the distance to revolutions and converts it to a 3 digit format used by the motor controller communication protocol. This 3 digit format is then input into a string and the command is sent through to the motor controller. Following this the function enters a waiting state until it receives the signal that robot has travelled the specified distance.

### 3.6.3 Turn degrees

The turn degrees function takes a value of direction in the form of 'l' and 'r' and a distance in degrees. The number of revolutions required to reach these degrees is calculated through multiplying it by a constant that was calibrated though testing in the environment. The number of revolutions is then converted to a 3 digit format as required by the motor controller. Once the message has been sent the function enters a waiting state until it receives the signal that the robot has travelled to the correct angle.

## 3.7 Chassis Design

The final part of the design process was ensuring that the chassis required for the mobility system was compatible with the rest of the subsystems. To achieve this it was decided that the chassis would be 3d printed to allow for any and all complex geometry to be produced in a cost-effective manner. The design of the overall chassis was left mostly in the hands of item collection as the scissor lift design required integration on all elements of the chassis. As such the impact of the mobility subsystem was minimal with the only real design influence being the inclusion of 2 lowered sections to raise the robot off the ground to ensure that the bottom didn't collide with any aspect of the warehouse (see figure 2).

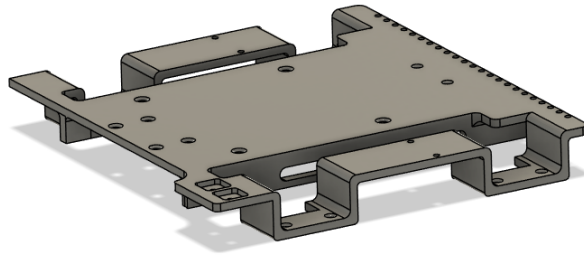


Figure 2: (   
Figure 2: Mobility contribution to chassis design)



### 3.8 Design Validation

The mobility system was validated through meeting various targets, these targets included:

- Be able to move at a speed greater than 0.3m/s
- Be able to move at a constant speed less than 0.05m/s
- Be able to travel up a 20° slope with a total weight of 1kg
- Be able to spin in place to a particular angle

These system specific goals were tested individually throughout the development of the system quantitatively. In the cases involving straight line speed a speed gate was used to get the most accurate ready through the use of laser detectors. In the case of turning on a spot a circle the size of the robot was drawn, and a top-down video was recorded to evaluate if the robot left the circle at any point. For the incline test weights were added to the robot until it weighed 1kg and a 2° incline was constructed.

The mobility subsystem met all requirements except for the ability to move at a speed greater than 0.3 m/s. With the requirement for the high torque N20 motors speed had to be sacrificed to ensure that the system would perform reliably. Further testing of the integration code was performed alongside navigation development to ensure that all functions operated as expected. This also showed success as the robot was capable of accurately traversing the warehouse without any issues related to motor function.

## 4 Conclusion

Overall the mobility subsystem utilizes tracks to produce robot movement, this is due to the reliability of tracks in regard to accurate turning and avoiding wheel slip. The tracks are turned by N20 high torque encoded motors that are powered by the Pimoroni Pico Motor 2040 motor controller. The Pimoroni Pico Motor 2040 takes commands over a USB that is then interpreted and acted upon according the configured communication protocol. The motor controller then employs the use of the magnetic encoders build into each motor to detect both speed and position, using this data and PID models for each it accurately controls the motors to ensure that the tasks set forth by the navigation subsystem are carried out. The main power delivery for the robot is done through a DCDC Buck converter that is run into the Raspberry Pi computer responsible for navigation. The motor controller however takes the raw input and uses that as the driving power for motors to ensure that there is no overload on the DCDC Buck converter. Throughout the development process the mobility subsystem has been constantly validated to ensure that it meets the requirements to complete the tasks expected of it. As such it is deemed that the mobility subsystem is capable of driving at a set speed or distance, turning to a specific angle on a point, deliver power to the rest of the robot reliably and have a chassis that houses all other subsystems in a way that they are protected.